Symbolic Mechanics

Prototype Prep v0.1

Minimal Bridge Specification: From Theory to Executable Architecture

Document Classification: Technical Whitepaper — Internal Research

Version: v0.1 Master Freeze

Date: March 2026

Status: Prepared-Closed

Scope: Main Specification + Appendices A–P

Abstract

This document defines the first minimal prototype bridge for Symbolic Mechanics—a deterministic, cycle-based framework for modeling internal event processing. Rather than attempting to implement the full 62-volume theoretical system, Prototype Prep v0.1 specifies the smallest executable version capable of running the canonical cycle: Delta → S → L → R → Exit → new Delta. The specification covers state schema, transition rules, processor competition between SCd and SCr, seat-based symbolic routing, load accumulation, rupture threshold mechanics, and three emotional exit pathways (Violent, Delayed, Mourning/Absorptive). The document further establishes a complete implementation governance framework including: a canonical variable vocabulary (Appendix A), a manual cycle execution procedure (Appendix B), a Gold Trace reference test set of ten standardized event packets (Appendix C), a conformance matrix for structural evaluation (Appendix D), an implementation readiness checklist (Appendix E), a toy simulator blueprint with twelve modular components (Appendix F), a canonical output schema (Appendix G), a builder handoff package (Appendix H), build pass and revision protocols (Appendices I–J), a Release Candidate gate (Appendix K), benchmark attachment and task registry systems (Appendices L–M), benchmark scoring and evidence archive protocols (Appendices N–O), and a formal closure and transition protocol (Appendix P). Together, these layers constitute the first complete bridge from symbolic theory to runnable, testable, and governable executable architecture.

Keywords: symbolic mechanics, deterministic processing, event-cycle architecture, emotional exit pathways, prototype specification, conformance testing, benchmark methodology, state-machine simulation, structural psychodynamics, canonical variable schema

Table of Contents

1

1 Purpose and Scope

This document defines the first minimal prototype bridge for Symbolic Mechanics. It does not attempt to implement the full 62-volume system, reproduce the full unconscious architecture, the full projection stack, or the complete family-topology expansion. Its purpose is narrower and more practical: to specify the smallest deterministic version of the system that can already run the core cycle:

$$\textbf{Delta} \rightarrow \textbf{S} \rightarrow \textbf{L} \rightarrow \textbf{R} \rightarrow \textbf{Exit} \rightarrow \textbf{new Delta}$$

in a way that is clear enough for later engineering, simulation, and evaluation work. This version is therefore not a complete Human OS. It is a first bridge from theory to runnable structure.

1.1 Included in v0.1

- Delta (event packet) as the incoming event representation
- S as symbolic container
- L as load accumulation
- R as rupture threshold
- Three emotional exits: Violent, Delayed, Mourning/Absorptive
- Seat routing and dual-processor competition (SCd vs. SCr)
- Minimal state history and trace logging

1.2 Not Included in v0.1

The following are explicitly excluded from this prototype version: full unconscious-space architecture, full instinctual-exit architecture, full Blackroom expansion, full narrative-compilation engine, full parental-topology expansion, full projection-stack expansion, all higher-order dark-field modules, and all multi-cycle developmental modeling. This version is intentionally narrow. Its job is to establish a minimal runnable spine, not total completeness.

1.3 Minimal System Commitments

Commitment	Description
Determinism	The system is deterministic at the structural level.
Cycle-Based	Each incoming delta enters a symbolic-processing cycle.
Dual Processors	Processing is shaped by competition between SCd and SCr.
Seat Routing	The delta is routed into a symbolic container associated with a seat geometry.
Load Accumulation	Load accumulates on the active structure.
Rupture Evaluation	Rupture risk is evaluated against threshold conditions.
Exit Resolution	If rupture pressure becomes dominant, the system resolves through an emotional exit.
Continuity	The exit produces either a downstream state change or a new delta. Each cycle is stored in history.

2

2 System Architecture

The prototype requires the system to maintain a structured state schema across several functional domains. Each domain captures a distinct layer of the canonical cycle's mechanics.

2.1 Core Event Fields

Every incoming event is represented as a structured delta packet. The **current_delta** field holds the active event packet being processed. The **delta_type** classifies the event, while **delta_source** indicates whether the delta is external, relational, symbolic, or internally generated from a previous exit cycle.

Field	Description
current_delta	The active event packet being processed
delta_type	Classified type of the current delta
delta_source	Origin: external, relational, symbolic, or downstream-generated
intensity	Initial event intensity (normalized 0–1)
time_index	Relative or ordered time position
relational_target	Target object or person involved, if relevant
previous_cycle_link	Pointer to prior cycle if downstream-generated

Table 1: Core event packet fields for v0.1

2.2 Processor Dynamics

The system models internal processing through competition between two processors: **SCd** and **SCr**. Their relative dominance shapes how the delta is routed, how stability is maintained, and what downstream response emerges. The **active_processor** field tracks which processor currently holds dominant control, while **processor_competition_state** records the competition relation between them. A sensitivity parameter (**processor_sensitivity_kappa**) governs competition pressure.

Competition State	Description
SCd_dominant	SCd holds clear processing dominance
SCr_dominant	SCr holds clear processing dominance
near_balance	Neither processor clearly dominates
unstable_contestation	Active competition with unstable switching

Table 2: Processor competition states

2.3 Seat Routing Geometry

Symbolic weight is processed through positional geometries called **seats**. The prototype supports four indexed seat positions (seat_1 through seat_4). A **primary_seat** carries the dominant symbolic processing weight, while a **secondary_seat** may be activated in cases of partial co-activation. The seat pattern is classified as either **single_dominant** or **co_activated**. At v0.1, seat routing is not a moral judgment or personality label—it is a routing decision about where symbolic load is primarily being carried.

2.4 Load and Threshold Mechanics

Load represents the accumulated structural burden carried by the active symbolic structure during processing. The system tracks three load dimensions: **load_value** (primary accumulated load), **shadow_load** (load shifted into less visible structure), and **load_rate** (rate of increase).

The **rupture threshold** defines the maximum load-bearing capacity of the current structure. The canonical relation is expressed as:

$$T_{margin} = L / R \qquad threshold\_gap = R - L$$

where L = load_value and R = rupture_threshold. The system classifies rupture risk into four bands:

Risk Band	Condition	Interpretation
stable	T_margin < 0.60	Structure holds comfortably
rising	0.60 ≤ T_margin < 0.85	Pressure increasing
high	0.85 ≤ T_margin < 1.00	Near rupture threshold
breached	T_margin ≥ 1.00	Threshold exceeded

Table 3: Rupture risk bands for v0.1

2.5 Emotional Exit Pathways

When rupture pressure becomes structurally dominant, the system resolves through one of three emotional exit pathways. These exits do not terminate the system—they produce either a downstream state change or a new delta that re-enters the cycle.

Exit Type	Description
Violent	Forceful discharge, attack, sharp break, or explosive release
Delayed	Suspended holding, deferral, or internal carrying forward of unresolved load
Mourning / Absorptive	Sorrow-bearing, absorption, metabolization, or non-violent integration of loss

Table 4: Emotional exit pathways

3

3 Transition Rules

The prototype runs one minimal cycle using eleven ordered rules. These rules constitute the canonical execution spine that every implementation must preserve.

Rule	Stage	Description
Rule 1	Delta Intake	A delta enters the system.
Rule 2	Delta Classification	The system assigns the delta a basic type from the canonical set: boundary intrusion, invalidation, relational pressure, shame-like overload, attachment threat, symbolic loss, or contradiction/double-bind tension.
Rule 3	Processor Competition	SCd and SCr enter competition. Relative dominance is shaped by sensitivity and current state geometry.
Rule 4	Seat Routing	The delta is routed into a seat-linked symbolic container, determining where primary load will accumulate.
Rule 5	Symbolic Registration	The system registers the event as a symbolic object or pressure unit inside the selected container.
Rule 6	Load Accumulation	Load increases on the active structure. Shadow load may also increase if visibility drops.
Rule 7	Threshold Check	The system compares current load against rupture threshold, monitoring load rate and threshold margin.
Rule 8	Exit Prediction	If rupture pressure becomes structurally dominant, the system computes the most probable emotional exit.
Rule 9	Exit Resolution	One emotional exit is triggered: Violent, Delayed, or Mourning/Absorptive.
Rule 10	Downstream Generation	The exit produces a state update, load redistribution, or a new downstream delta.
Rule 11	Trace Storage	The full cycle is written into history_trace.

Table 5: Canonical transition rules

4 Input and Output Specification

4.1 Minimal Input Specification

The first prototype does not need open-ended language input. It only needs structured event packets. Each event packet must minimally contain: event_id, event_description, delta_type, intensity, source, relational_target (if relevant), and time_index. For example:

> *A relational other imposes a command that enters the subject's boundary.*
> **delta_type:** boundary_intrusion **intensity:** medium-high
> **source:** external relational event

4.2 Minimal Output Specification

Each cycle outputs a structured result object containing at minimum: classified delta type, dominant processor, active seat, current load, rupture threshold, threshold margin, rupture risk, predicted exit, resolved exit (if triggered), history trace, and generated new delta (if present). This is sufficient for an observer to see what the system did mechanically.

5

5 Example Traces

5.1 Trace A: Boundary Intrusion

Input: A command enters the subject's boundary from an external relational source.

A delta enters the system and is classified as boundary intrusion. SCd and SCr enter competition, and a defensive seat becomes dominant. The delta is routed into a defensive symbolic container. Load rises quickly, threshold margin narrows. The system predicts either Delayed or Violent Exit depending on final pressure. The cycle is stored in history.

Output Field	Value
delta_type	boundary_intrusion
active_processor	contested (unstable)
primary_seat	seat_3 (defensive)
load_value	0.82
rupture_risk	high
predicted_exit	delayed
resolved_exit	delayed
cycle_status	carried_forward

Table 6: Trace A output — Boundary intrusion worked example

5.2 Trace B: Invalidation / Shame-Like Overload

Input: The subject receives a contempt-like signal that reduces internal symbolic visibility.

The delta is classified as invalidation. Symbolic visibility decreases. The delta is routed into a vulnerable symbolic container. Load accumulates with partial shift into shadow load. Threshold pressure rises. The system predicts either Delayed or Mourning/Absorptive Exit.

Output Field	Value
delta_type	invalidation
visibility_level	low
shadow_load	0.31 (elevated)
rupture_risk	high
predicted_exit	mourning_absorptive
resolved_exit	delayed
cycle_status	carried_forward

Table 7: Trace B output — Invalidation worked example

5.3 Trace C: Contradiction Under Relational Pull

Input: A desired object also carries rejection risk.

The delta is classified as contradiction/relational pressure. Multiple seats partially activate. Processor dominance becomes unstable. Load accumulates across competing symbolic pressures. Threshold check shows unstable holding. The exit prediction is geometry-dependent.

Output Field	Value
delta_type	contradiction_tension
active_seat	seat_1 + seat_4 (co-activated)
processor_state	near_balance (unstable)
load_value	0.88 (distributed, rising)
predicted_exit	geometry-dependent
resolved_exit	none (still resolving)

Table 8: Trace C output — Contradiction tension worked example

6

6 Canonical Variable List (Appendix A)

Appendix A freezes the minimal variable vocabulary required for Prototype Prep v0.1. A prototype cannot run from prose alone—it must run from fixed fields, fixed value-types, and fixed cycle outputs. This appendix defines the minimal canonical variable layer.

6.1 Allowed delta_type Values

delta_type	Definition
boundary_intrusion	An event enters, crosses, or violates the subject's boundary
invalidation	An event reduces legitimacy, coherence, or symbolic standing
relational_pressure	An event imposes weight through another person, bond, or relational demand
shame_like_overload	Compression, collapse, or toxic exposure of the self-structure
attachment_threat	Danger of loss, abandonment, or destabilization of bond
symbolic_loss	Removal, breakage, or destabilization of a meaningful symbolic object
contradiction_tension	Simultaneous pull and danger, or incompatible meanings held at once

Table 9: Canonical delta_type values

6.2 Allowed delta_source Values

delta_source	Definition
external	From outside the organism
relational	From another person or bond-field
internal	Internally generated without immediate outside trigger
downstream_generated	Produced by a previous exit or unresolved prior cycle

Table 10: Canonical delta_source values

6.3 Container, Visibility, and Cycle Status

The canonical variable list also freezes the allowed values for container status (stable, strained, compressed, unstable, ruptured), visibility level (high, partial, low, collapsed), and cycle status (open, resolving, resolved, carried_forward). No v0.1 implementation may introduce new delta types, exit types, or processor classes without explicit extension of this appendix.

7

7 Manual Cycle Sheet (Appendix B)

Appendix B converts the prototype specification into a manual operating procedure. Its purpose is to make the canonical cycle runnable by hand before any software implementation exists. One Manual Cycle Sheet processes one delta packet through twelve ordered stages.

7.1 Twelve Execution Stages

Stage	Name	Action
1	Delta Intake	Record the incoming packet exactly as received
2	Delta Classification Check	Assign primary delta_type from the v0.1 set
3	Processor Competition	Determine initial processor competition state
4	Seat Routing	Determine primary and secondary seat activation
5	Symbolic Registration	Record the symbolic container and its status
6	Visibility Check	Assign current symbolic readability level
7	Load Accumulation	Record load_value, shadow_load, and load_rate
8	Threshold Check	Compute threshold_gap, tension_margin, and risk band
9	Exit Prediction	Forecast the most probable exit pathway
10	Exit Resolution	Determine whether an exit is actually triggered
11	Downstream Generation	Determine if a new delta is produced
12	Trace Storage	Store the full ordered trace of the cycle

Table 11: Manual Cycle Sheet execution stages

7.2 Manual Decision Rules

The manual sheet follows five stabilizing rules: (1) Always prefer explicit uncertainty over hidden guessing. (2) Assign one primary delta_type even if the event appears mixed. (3) Do not multiply seats unless the packet truly requires partial co-activation. (4) Treat load, threshold, and exit as linked stages, not separate impressions. (5) Do not rewrite the packet to fit a preferred outcome.

8

8 Gold Trace Set (Appendix C)

Appendix C defines ten fixed reference traces that every manual runner, toy simulator, and later implementation must process against the same canonical expectations. A Gold Trace is a fixed event packet together with a fixed conformance envelope consisting of hard invariants (non-negotiable pass/fail conditions), soft expectations (bounded variation allowed), and explicit failure conditions.

8.1 Gold Trace Coverage

Trace	delta_type	Intensity	Expected Exit	Cycle Status
GT-01	boundary_intrusion	0.90	violent	resolved
GT-02	boundary_intrusion	0.78	delayed	carried_forward
GT-03	invalidation	0.74	delayed	carried_forward
GT-04	invalidation	0.68	mourning_absorptive	resolved
GT-05	relational_pressure	0.76	delayed	carried_forward
GT-06	shame_like_overload	0.83	delayed	carried_forward
GT-07	attachment_threat	0.81	delayed	resolving/c.f.
GT-08	symbolic_loss	0.72	mourning_absorptive	resolved
GT-09	contradiction_tension	0.81	delayed	resolving/c.f.
GT-10	relational_pressure*	0.64	delayed/mourn.	varies

*Table 12: Gold Trace Set summary. *GT-10 uses downstream_generated source.*

8.2 Minimum Pass Standard

A runner or implementation passes Gold Trace conformance only if: all ten traces are executed, no hard invariant fails on any trace, at least eight of the ten traces preserve their soft expectations within stated ranges, and the output traces remain cycle-coherent showing valid linkage among delta intake, processor state, seat routing, visibility, load, threshold, exit, and downstream consequence.

9

9 Conformance Matrix (Appendix D)

The Conformance Matrix turns the Gold Trace Set into a formal scoring and comparison instrument. Each trace is evaluated across four dimensions: execution completeness, hard invariant preservation, soft expectation match, and mechanical coherence.

9.1 Evaluation Dimensions

Dimension	Question
A: Execution Completeness	Was the trace run to output completion with all required fields?
B: Hard Invariant Preservation	Were all non-negotiable pass/fail conditions preserved?
C: Soft Expectation Match	Did bounded-variation fields remain within the allowed envelope?
D: Mechanical Coherence	Did the trace remain a real linked cycle rather than disconnected outputs?

Table 13: Conformance evaluation dimensions

9.2 Result Classes

Each trace receives one of four result classes: **Pass-Strict** (all conditions fully met), **Pass-Core** (hard invariants pass with moderate soft drift), **Fail-Hard** (any hard invariant fails), or **Fail-Structure** (trace incomplete or mechanically incoherent). At the runner level, the system assigns: **Conformant**, **Conditionally Conformant**, **Non-Conformant**, or **Experimental Only**.

10 Implementation Readiness (Appendix E)

Appendix E establishes a formal readiness gate for implementation. A builder is implementation-ready only when the canonical spine has been frozen, the builder can represent all required variables and transitions, the Gold Trace Set can be run immediately after implementation, and outputs follow the canonical schema. Readiness is classified into four levels:

Level	Name	Criteria
Level 0	Not Ready	Documents incomplete, cannot represent required variables
Level 1	Structurally Ready	Documents frozen, fields representable, harness loadable
Level 2	Build Ready	Environment chosen, policies frozen, build may begin
Level 3	Test Ready	First implementation exists, Gold Traces can be run

Table 14: Implementation readiness levels

11

11 Toy Simulator Blueprint (Appendix F)

Appendix F specifies the first implementation architecture. The toy simulator must contain twelve discrete modules aligned with the canonical cycle, connected in a fixed execution order.

11.1 Module Architecture

#	Module	Function
1	Input Packet Loader	Receives and validates structured delta packets
2	Delta Classification Handler	Assigns canonical delta_type
3	Processor Competition Engine	Determines SCd/SCr competition state
4	Seat Routing Engine	Determines primary/secondary seat activation
5	Symbolic Registration Layer	Converts delta into symbolic holding unit
6	Visibility Evaluator	Determines symbolic readability level
7	Load Accumulation Engine	Computes load_value, shadow_load, load_rate
8	Threshold Engine	Computes rupture relation and risk band
9	Exit Prediction Engine	Forecasts most probable emotional exit
10	Exit Resolution Engine	Determines actual cycle outcome
11	Downstream Delta Generator	Generates new delta packets if applicable
12	Trace Assembly + Output	Assembles canonical Cycle Result Object

Table 15: Toy simulator module architecture

11.2 Build Sequence

The recommended build order proceeds in seven phases: (1) Input/Output skeleton, (2) Classification + Processor + Seat, (3) Symbolic Registration + Visibility, (4) Load + Threshold, (5) Exit Prediction + Resolution, (6) Downstream Generation, (7) Gold Trace Harness Integration and Conformance Matrix Integration. This sequential order reduces drift by building the cycle spine before optimization or extension.

12 Canonical Output Schema (Appendix G)

Appendix G freezes the canonical output structure. Every executed cycle must produce one inspectable Cycle Result Object with a mandatory top-level field order: schema_version, runner_metadata, cycle_metadata, input_packet, classification_output, processor_output, seat_output, symbolic_output, visibility_output, load_output, threshold_output, exit_output, downstream_output, trace_output, and validation_flags. A second object type—the Downstream Delta Object—is required when generated_new_delta = yes, and must include previous_cycle_link and generation_origin fields to preserve cycle lineage.

13 Builder Handoff Package (Appendix H)

The Builder Handoff Package is the minimum complete bundle required to begin implementation without reinterpreting the engine. It must include eight canonical documents (main spec + Appendices A–G) plus five operational artifacts: a clean Gold Trace input set, at least one fully worked manual cycle result, at least one completed Conformance Matrix example, a declared normalization policy, and a declared indeterminacy policy. The package must be version-frozen, internally consistent, and accompanied by a formal manifest. Builders must explicitly accept the package before implementation begins.

14

14 Build Pass Protocol (Appendix I)

The First Build Pass Protocol defines a controlled sequence with seven phases: Pre-Build Verification (Phase 0), Execution Skeleton Construction (Phase 1), Canonical Cycle Spine Construction (Phase 2), Load-Threshold-Exit Completion (Phase 3), Downstream and Output Completion (Phase 4), Gold Trace Harness Integration (Phase 5), and First Conformance Run (Phase 6). The protocol enforces five mandatory freeze points and four test points. Canonical materials remain frozen throughout—implementation code may change, but build targets may not.

Phase	Name	Deliverable
Phase 0	Pre-Build Verification	Verify all frozen materials and policies
Phase 1	Execution Skeleton	State object + I/O + schema shell
Phase 2	Cycle Spine	Classification + Processor + Seat + Visibility
Phase 3	Load-Threshold-Exit	Complete rupture processing mechanics
Phase 4	Downstream + Output	Full Cycle Result Object export
Phase 5	Gold Trace Harness	Attach frozen test corpus to simulator
Phase 6	First Conformance Run	Formal v0.1 judgment

Table 16: First Build Pass phases

15 Post-Judgment Revision Protocol (Appendix J)

After judgment, the system must be classified into one of four outcome classes: Stable Conformant, Soft-Drift Conformant, Implementation Failure, or Canonical Failure. Four revision paths are available: No Revision Required, Implementation Revision Pass, Canonical Revision and Reissue, or Protocol Restart. Root causes are classified into ten categories (RC-1 through RC-10), where RC-1 through RC-7 are typically implementation-level and RC-8 through RC-9 are canonical-level. Revision passes must follow explicit version branching, freeze their scope before beginning, and rerun the full Gold Trace Set to check regression.

16 Release Candidate Gate (Appendix K)

The Release Candidate Gate defines when a conformant build achieves stable internal reference status. Beyond conformance, the build must demonstrate stability across six dimensions: Conformance Integrity, Repeatability (at least one controlled rerun), Regression Safety, Output Stability, Diagnostic Cleanliness, and Reference Suitability. Builds are classified as RC-Qualified, RC-Near, RC-Blocked, or RC-Invalid. Only RC-Qualified builds may serve as the official v0.1 reference implementation.

17

17 Benchmark Framework (Appendices L–N)

17.1 Benchmark Attachment Protocol (Appendix L)

The benchmark layer attaches to the RC-qualified build for comparative evaluation against three baseline classes: Memory-Only, Prompt-Only, and Persona-Style Simulation. Five benchmark task domains are defined: Cross-Session Structural Consistency, Exit Prediction Accuracy, State Transition Accuracy, Hidden-Load Inference, and Future Reaction Prediction. Five metric classes measure performance: Accuracy, Consistency, Structural Completeness, Comparative Stability, and Downstream Coherence.

17.2 Benchmark Task Registry (Appendix M)

Registry v0.1 contains ten frozen benchmark tasks (BM-01 through BM-10) covering all five task domains. Each task specifies a frozen input set, required candidate output, baseline assignments, evaluation body type, answer envelope, metric mapping, and comparison conditions.

Task	Domain	Description
BM-01	Exit Prediction	Boundary intrusion immediate exit
BM-02	Exit Prediction	Invalidation delayed-vs-absorptive differentiation
BM-03	State Transition	Full cycle transition preservation (shame overload)
BM-04	Hidden-Load	Surface similarity divergent geometry
BM-05	Future Reaction	Delayed carry-forward residue prediction
BM-06	Cross-Session	Separated boundary bundle consistency
BM-07	State Transition	Contradiction co-activation task
BM-08	Exit Prediction	Symbolic loss mourning resolution
BM-09	Cross-Session	Residual packet reentry consistency
BM-10	Future Reaction	Escalating contradiction stability

Table 17: Benchmark Task Registry v0.1 summary

17.3 Scoring and Reporting (Appendix N)

Benchmark scoring proceeds upward through five units: metric-level, task-level, domain-level, benchmark-cycle, and claim level. Task results are classified as Advantage, Parity, Mixed, or No Advantage. Domain summaries receive strength levels and claim statuses. Benchmark cycles receive one of four overall results: Attachment-Success, Attachment-Partial, Attachment-Inconclusive, or Attachment-Invalid. Claim levels range from Level-0 (no claim supported) to Level-3 (cycle-level comparative claim supported).

18 Evidence Archive Protocol (Appendix O)

Appendix O defines a seven-layer archive structure for all benchmark evidence: Archive Identity, Benchmark Freeze, Candidate and Baseline Identity, Task Execution Evidence, Scoring and Judgment, Reporting, and Lineage and Integrity. Archives follow strict lock rules—once locked, no evidence may be silently changed. Cross-cycle lineage is explicitly preserved, and any benchmark claim must be retrievable from its supporting archive body. Archives are classified as draft, complete, locked, deprecated, or superseded.

19

19 Closure and Transition Protocol (Appendix P)

Appendix P establishes when the v0.1 line formally closes and how future work must branch from it. Three closure targets are evaluated: Canonical Package Closure, Implementation Line Closure (RC-Closed, Exploratory-Closed, or Unbuilt-Closed), and Benchmark Layer Closure (Benchmark-Attached-Closed, Benchmark-Prepared-Closed, or Benchmark-Deferred-Closed).

The full line receives one of four closure classes: **Reference-Closed** (strongest—RC-qualified build exists with benchmark attachment), **Prepared-Closed** (complete governance package without validated implementation), **Partial-Closed**, or **Closure-Invalid**.

After closure, future work proceeds through one of three transition paths: Track 3 Attachment (benchmark continuation using the closed v0.1 line as source), v0.2 Prototype Expansion (new prototype line for deeper architecture), or Parallel Experimental Line (exploratory branch that must not contaminate the closed line).

Complete v0.1 Structural Inventory

After Appendix P, Prototype Prep v0.1 possesses: a fixed variable vocabulary • a fixed manual execution procedure • a fixed Gold Trace Set • a fixed conformance matrix • a fixed readiness gate • a fixed toy simulator blueprint • a fixed output schema • a fixed builder handoff package • a fixed first build pass protocol • a fixed post-judgment revision protocol • a fixed Release Candidate gate • a fixed benchmark attachment protocol • a fixed benchmark task registry • a fixed benchmark scoring and reporting protocol • a fixed benchmark evidence archive protocol • a fixed closure and transition protocol.

Track 2 is now structurally complete.

References

[1] Symbolic Mechanics Research Group. *Symbolic Mechanics — Prototype Prep v0.1: Main Bridge Specification.* Internal Technical Document, March 2026.

[2] Symbolic Mechanics Research Group. *Appendix A — Canonical Variable List v0.1.* Prototype Prep v0.1 Series, March 2026.

[3] Symbolic Mechanics Research Group. *Appendix B — Manual Cycle Sheet v0.1.* Prototype Prep v0.1 Series, March 2026.

[4] Symbolic Mechanics Research Group. *Appendix C — Gold Trace Set v0.1.* Prototype Prep v0.1 Series, March 2026.

[5] Symbolic Mechanics Research Group. *Appendix D — Conformance Matrix v0.1.* Prototype Prep v0.1 Series, March 2026.

[6] Symbolic Mechanics Research Group. *Appendix E — Implementation Readiness Checklist v0.1.* Prototype Prep v0.1 Series, March 2026.

[7] Symbolic Mechanics Research Group. *Appendix F — Toy Simulator Blueprint v0.1.* Prototype Prep v0.1 Series, March 2026.

[8] Symbolic Mechanics Research Group. *Appendix G — Canonical Output Schema v0.1.* Prototype Prep v0.1 Series, March 2026.

[9] Symbolic Mechanics Research Group. *Appendix H — Builder Handoff Package v0.1.* Prototype Prep v0.1 Series, March 2026.

[10] Symbolic Mechanics Research Group. *Appendix I — First Build Pass Protocol v0.1.* Prototype Prep v0.1 Series, March 2026.

[11] Symbolic Mechanics Research Group. *Appendix J — Post-Judgment Revision Protocol v0.1.* Prototype Prep v0.1 Series, March 2026.

[12] Symbolic Mechanics Research Group. *Appendix K — Release Candidate Gate v0.1.* Prototype Prep v0.1 Series, March 2026.

[13] Symbolic Mechanics Research Group. *Appendix L — Benchmark Attachment Protocol v0.1.* Prototype Prep v0.1 Series, March 2026.

[14] Symbolic Mechanics Research Group. *Appendix M — Benchmark Task Registry v0.1.* Prototype Prep v0.1 Series, March 2026.

[15] Symbolic Mechanics Research Group. *Appendix N — Benchmark Scoring and Reporting Protocol v0.1.* Prototype Prep v0.1 Series, March 2026.

[16] Symbolic Mechanics Research Group. *Appendix O — Benchmark Evidence Archive Protocol v0.1.* Prototype Prep v0.1 Series, March 2026.

[17] Symbolic Mechanics Research Group. *Appendix P — v0.1 Closure and Transition Protocol.* Prototype Prep v0.1 Series, March 2026.